Princípios de Entrada e Saída em Nível de Software

Professor: Paulo Roberto de Oliveira

Disciplina: Sistemas Operacionais

Departamento de Informática
Universidade Estadual de Maringá

Metas do Software de I/O

Independência de dispositivo:

- Programas podem usar arquivos em discos ou disquetes da mesma maneira!

Nomes uniformes:

- Os nomes de arquivos devem ser strings ou inteiros;
- Esses nomes não podem depender de onde os arquivos estão armazenados.

Metas do Software de I/O

Tratamento de erros:

- Erros devem ser tratados o mais perto possível do hardware;
- Se a controladora descobre um erro de leitura, ela deve tentar corrigi-lo sozinha;
- Em geral, ela tenta ler novamente;
- Muitos erros são *transientes* - ex. problemas elétricos no momento da leitura - desaparecem se a operação é repetida.

Metas do Software de I/O

Buffers:

- É comum ter que armazenar um dado antes que ele seja entregue no destino (pacotes de rede);
- Alguns dispositivos têm restrições de tempo que obrigam a guardar dados em buffers para atender as taxas de transmissão (som e vídeo);
- O uso de buffers implica em tempo para copiar dados de um lado para outro.

Metas do Software de I/O

Dispositivos compartilhados x exclusivos:

- Alguns dispositivos (ex. disco) podem ser usados por mais de um usuário ao mesmo tempo;
- Outros (ex. fitas) devem ser dedicados;
- Dispositivos dedicados trazem problemas, tipo deadlocks.

I/O Programado

- Forma mais simples de se fazer I/O, onde a CPU é responsável por tudo.

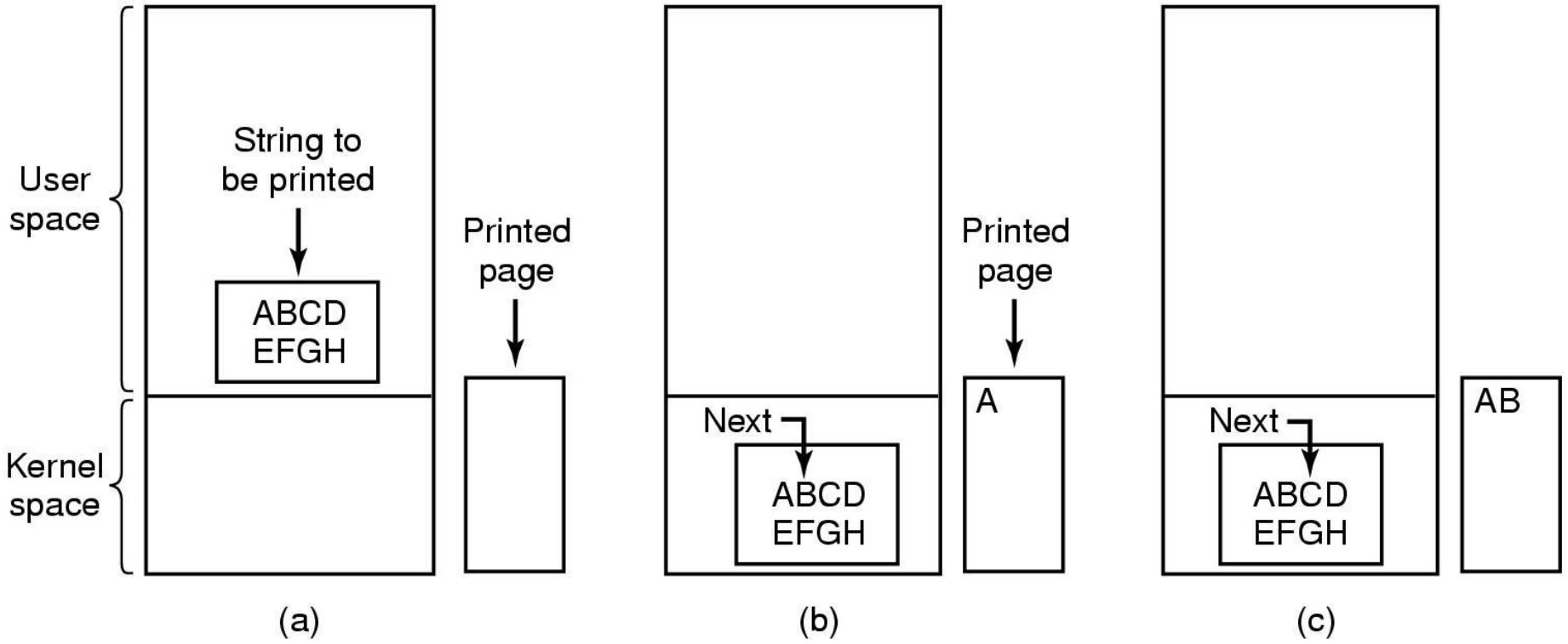
<u>Exemplo</u>:

- Suponha um programa que deseja imprimir alguma coisa.
- O processo do usuário adquire a impressora (desde que esteja disponível) e faz uma chamada para imprimir o que deseja.
- O SO copia o texto para um buffer no espaço do kernel.

I/O Programado

- O SO verifica se a impressora está disponível (há um registrador que permite ver o estado dela).
- Se estiver, copia um caracter do buffer para o registrador de dados da impressora.
- Esta ação ativa a impressora.
- O SO verifica se a impressora está pronta para receber o próximo caracter e assim por diante.

I/O Programado



I/O Programado

- O que o SO faz pode ser visto abaixo:

```
copy_from_user(buffer, p, count);          /* p is the kernel bufer */
for (i = 0; i < count; i++) {              /* loop on every character */
      while (*printer_status_reg != READY) ;   /* loop until ready */
      *printer_data_register = p[i];       /* output one character */
}
return_to_user( );
```

- A 3ª. linha é uma operação de *busy waiting* (neste caso, também chamada de ***polling***).
- Quando a impressão estiver terminada, o sistema devolve o controle para o processo do usuário.

I/O Programado

- Este método é interessante se o tempo de impressão de um caracter for pequeno.
- Também é útil em sistemas embutidos, onde a CPU não tem mais nada para fazer.
- Sistemas mais complexos, onde há muitas atividades pendentes, precisam de outras soluções.

I/O Baseado em Interrupções

- Suponha uma impressora que imprime 100 caracteres/seg.
- Logo, a impressão de um caracter leva 10 mseg.
- Neste tempo (enquanto um caracter é imprimido), é possível fazer uma troca de contexto e processar outra coisa.

I/O Baseado em Interrupções

```
copy_from_user(buffer, p, count);
enable_interrupts( );
while (*printer_status_reg != READY) ;
*printer_data_register = p[0];
scheduler( );
```

(a)

```
if (count == 0) {
    unblock_user( );
} else {
    *printer_data_register = p[i];
    count = count – 1;
    i = i + 1;
}
acknowledge_interrupt( );
return_from_interrupt( );
```

(b)

I/O Baseado em Interrupções

- Neste caso, em (a) o tratamento da chamada de sistema dispara o primeiro caracter e chama o scheduler.
- Com interrupções habilitadas, quando a impressora imprimir um caracter ela gera uma interrupção.
- O serviço de interrupções verifica se os caracteres acabaram e desbloqueia o processo que pediu a impressão.
- Se não, envia outro caracter, confirma a interrupção para a impressora e devolve o controle ao processo interrompido.

I/O Usando DMA

- A desvantagem do método anterior é que as interrupções ocorrem a cada caracter.
- O tratamento de uma interrupção toma um certo tempo, então isso leva a queda de desempenho da CPU.
- Uma alternativa é o uso de DMA.

I/O Usando DMA

```
copy_from_user(buffer, p, count);
set_up_DMA_controller( );
scheduler( );
```

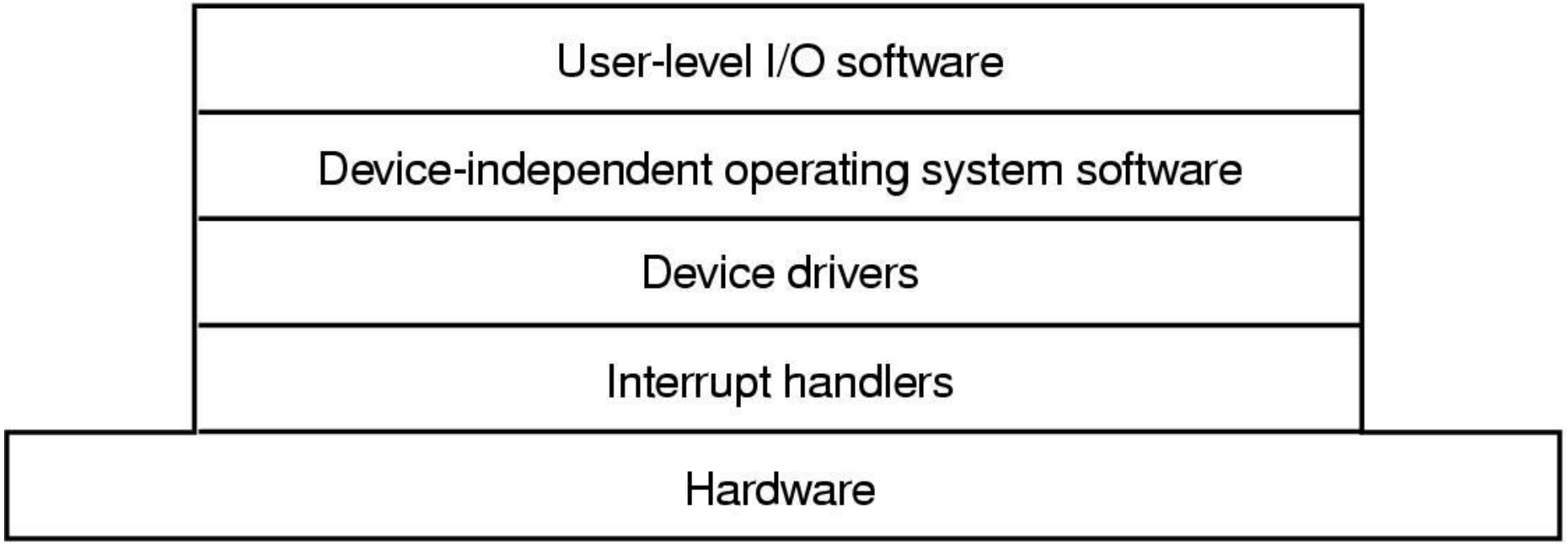
(a)

```
acknowledge_interrupt( );
unblock_user( );
return_from_interrupt( );
```

(b)

- Neste caso, o controlador de DMA é programado e a CPU vai fazer outra coisa.
- Apenas no fim, o tratamento de interrupção é avisado e o I/O está pronto.

Camadas de Software de I/O



Interrupt Handlers

- Identifica de qual dispositivo é a interrupção;
- Salva os registradores do processador;
- Cria um contexto sobre o dispositivo nos registradores do processador;
- Chama a rotina de tratamento do dispositivo;
- Retorna o contexto no processador para o processo anterior a interrupção continuar a execução;

Device Drivers

- Contém o código dependente do dispositivo;

- É o “núcleo” da rotina de tratamento da interrupção;

- Ex.: Device driver de um disco:
  - Sabe quantos registradores a controladora tem e para que servem;
  - Conhece todos os elementos do disco: setores, trilhas, cilindros, cabeças, movimento do braço, interleave, motor, etc.

Device Drivers

Função do device driver:

- Trata múltiplas requisições;
- Verifica as condições do dispositivo para realizar as operações;
- Preenche os registradores da controladora do dispositivo;
- Verifica (e corrige) erros;
- Passa dados para o software independente de dispositivo;
- Retorna informação de status ou de erro.

Software Independente de Dispositivo

Funções:

- Interface uniforme com os device drivers:
- O uso de interfaces diferentes para cada dispositivo tornaria o SO muito complexo;
- Ele teria que ser modificado para cada novo dispositivo;
- A interface significa o conjunto de funções disponíveis para “contactar” qualquer que seja o driver.

Software Independente de Dispositivo

Proteção:

- Os dispositivos aparecem no sistema de arquivos como objetos com nomes comuns;
- As regras de proteção de arquivos se aplicam aos dispositivos;
- O administrador do sistema pode estabelecer as permissões adequadas para cada dispositivo.

Software Independente de Dispositivo

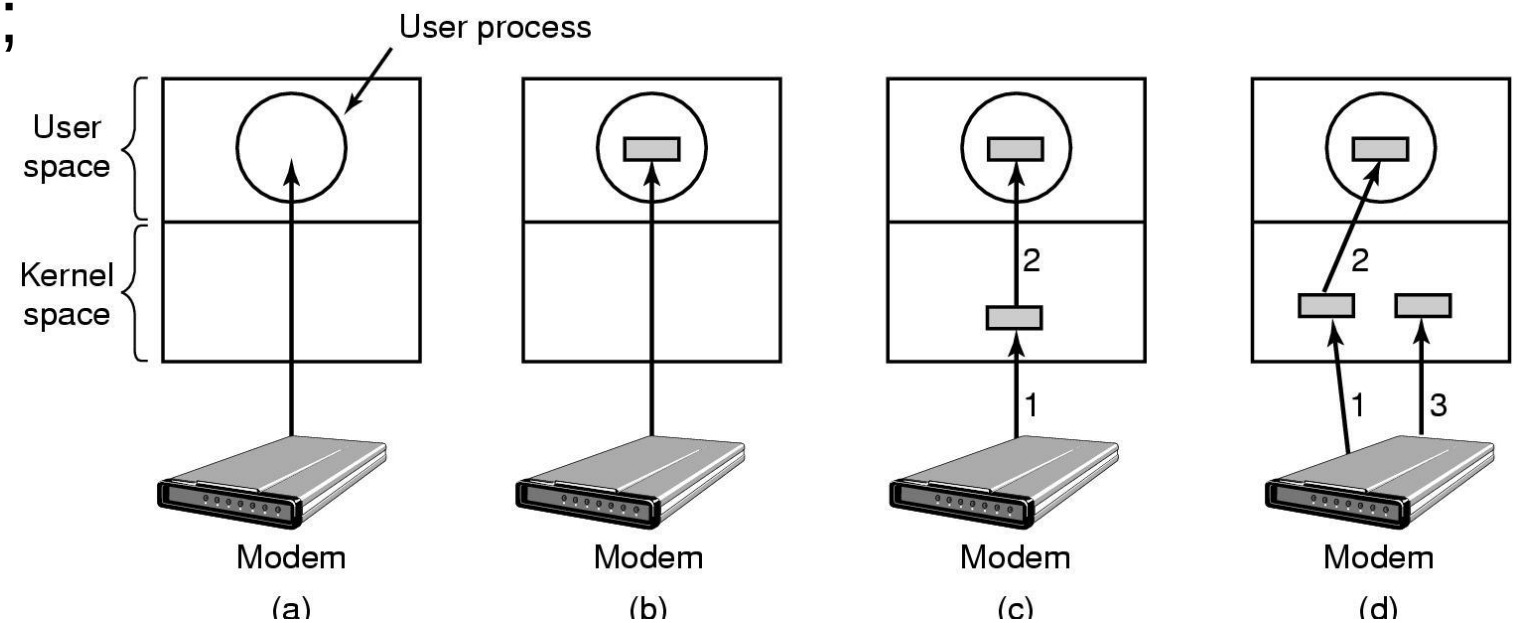
Tamanho de bloco independente de dispositivo:

- Discos diferentes podem ter diferentes tamanhos de bloco físico;
- O software independente de dispositivo pode criar um tamanho de bloco lógico para o sistema e resolver as diferenças entre as unidades específicas.

Software Independente de Dispositivo

Buffers:

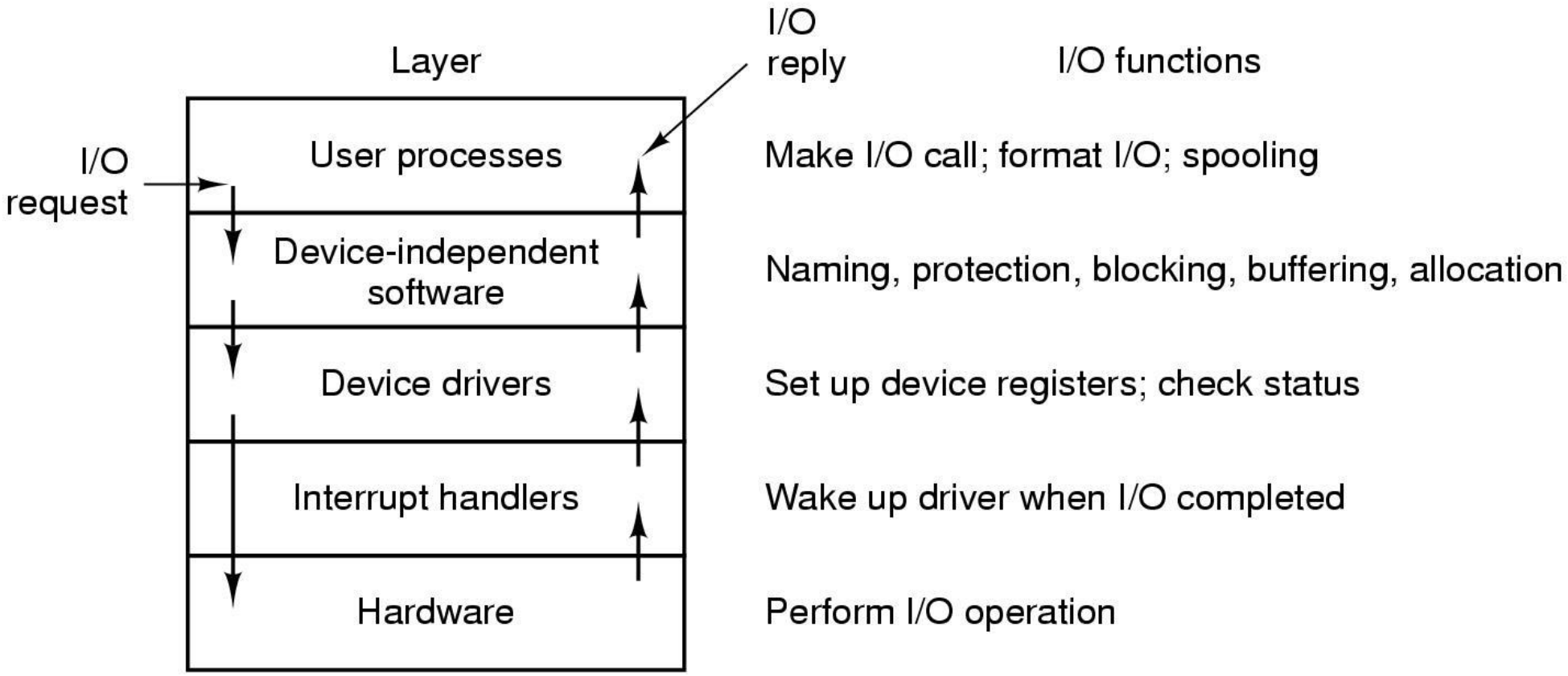
- O uso de buffers é extremamente importante em qualquer dispositivo;
- Na figura abaixo, pode-se ver o uso de um modem com e sem buffers;



Software Independente de Dispositivo

- Em (a) o processo do usuário recebe diretamente cada caracter;
- Em (b) o processo fornece um buffer no espaço do usuário para receber N caracteres – o processo é acordado quando o buffer está cheio;
- Em (c) o kernel fornece um buffer – quando ele está cheio, o modem é interrompido e o conteúdo é copiado para o usuário;
- Em (d) o kernel fornece 2 buffers – quando um está cheio, o modem pega o outro, enquanto o kernel transfere os dados recebidos até o momento.

Sumário do Sistema de I/O



Dúvidas?